Week 4: Branching, Looping, Conditional Statement, Exit function, Difference between break, continue and pass.

Decision making statements

Decision making statement used to control the flow of execution of program depending upon **condition**.

Python has the following decision-making statements:

- if statements
- if-else statements
- elif ladder
- Nested statements

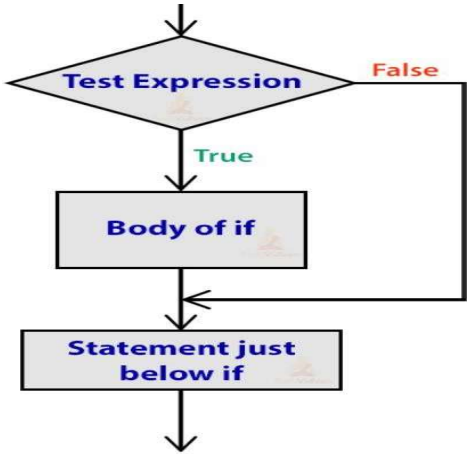
1. if statement

if statement is the simplest form of decision-making statement. It takes an expression and checks if the expression evaluates to True then the block of code in if statement will be executed. If the expression evaluates to False, then the block of code is skipped.

Syntax:

if (expression):
Statement 1
Statement 2
…
Statement n

Flowchart:



Note: Don't forget to add a colon(:) after if statement and indent the statements properly that are executed when a condition is True.

Example:

python program to illustrate If statement

age= 20

if (age >= 18):

  print ("Person is eligible for voting")

print ("This statement is out of if block")

Output:

Person is eligible for voting

This statement is out of if block

short Hand if statement

Whenever there is only a single statement to be executed inside the if block then shorthand if can be used. The statement can be put on the same line as the if statement.

Syntax:

if condition: statement

Example:

age= 20

if (age >= 18): print ("Person is eligible for voting")

print ("This statement is out of if block")

2. if-else statement

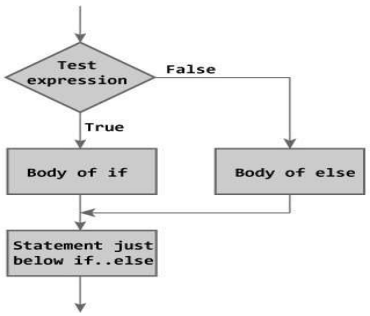
The if statement is used only if the task is performed when a condition is true. If a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else if the condition is false. Here comes the *else* statement. We can use the *else* statement with *if* statement to execute a block of code when the condition is false.

Syntax:

```
if (condition):
    True block statements(Body of if)
else:
    False block statements(body of else)
```

Flowchart:

Flowchart of if...else statement



Example:

```
# python program to illustrate if -else statement
age= 20
if (age >= 18):
   print ("Person is eligible for voting")
else:
   print ("Person is not eligible for voting")
print ("This statement is out of  if –else block")
```

output:

Person is eligible for voting

This statement is out of if-else block

Short Hand if-else statement

This can be used to write the if-else statements in a single line where there is only one statement to be executed in both if and else block.

Syntax:

statement_when_True **if** condition **else** statement_when_False

Example:

```
print ("Person is eligible for voting") if(age>=18) else print ("Person is not eligible for voting")
```

elif ladder:

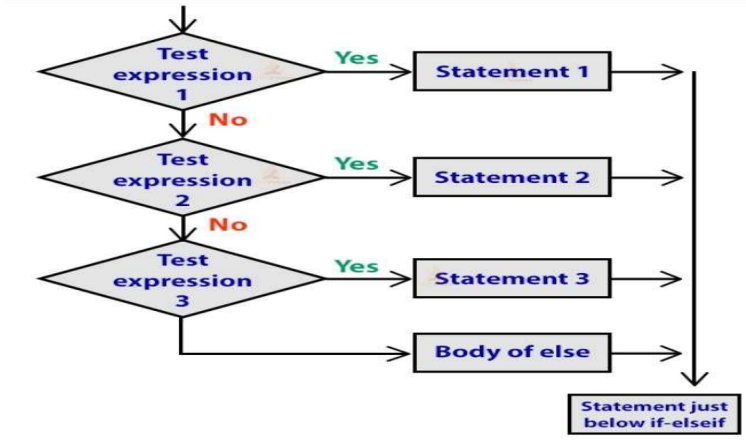
In Python, we have an elif keyword to chain multiple conditions one after another.

With elif ladder, we can make complex decision-making statements. The elif statement helps you to check multiple expressions and it executes the code as soon as one of the conditions evaluates to True.

Syntax:

```
if( expression1 ):
        statement
elif (expression2 ) :
        statement
elif(expression3 ):
        statement
.
.
else:
        statement
```

Flowchart:



Example:

```
i = 2
if (i == 1):
    print ("i = 1")
elif (i == 2):
    print ("i = 2")
elif (i == 3):
    print ("i = 3")
else:
    print ("value of i is not present")
```

Nested if-else

A nested if-else is an if –else statement that is the target of another if-else statement. Nested if –else statements means an if –else statement inside another if or else statement.

Syntax:

```
if (condition1):
  # Executes when condition1 is true
  if (condition2):
    # Executes when condition2 is true
```

```
    statement1
  else:
    # Executes when condition2 is false
    statement2
else:
  # Executes when condition1 is false
   if(condition3):
    statement3
  else:
    statement4
```

Example:

```
num1 = int( input("enter first number"))
num2 = int( input("enter second number"))
if( num1>= num2):
    if(num1 == num2):
        print("Both are equal")
   else:
       print(" num1>num2")
else:
    print("num1<num2")
```

Loops:

In general, statements are executed sequentially: The first statement is executed first, followed by the second, and so on. There may be a situation when you need to execute a block of code several number of times.

Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times.

A **while** loop statement in Python programming language repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a **while** loop in Python programming language is −

```
while expression:

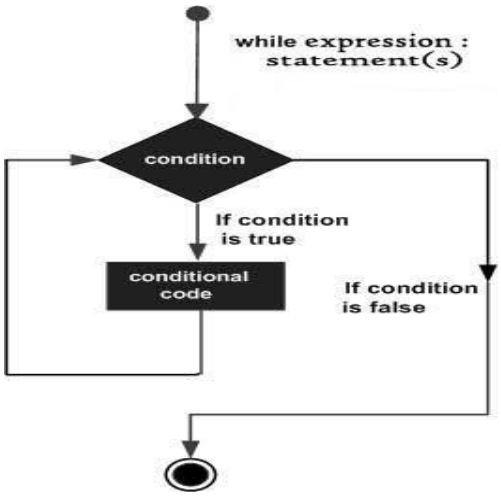
   statement(s)
```

Here, **statement(s)** may be a single statement or a block of statements. The **condition** may be any expression, and true is any non-zero value. The loop iterates while the condition is true.

When the condition becomes false, program control passes to the line immediately following the loop.

In Python, all the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

Flow Diagram



Here, key point of the while loop is that the loop might not ever run. When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example

```
count = 0
while (count < 9):
   print 'The count is:', count
   count = count + 1

print "End of program"
```

When the above code is executed, it produces the following result −

```
The count is: 0
The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5
The count is: 6
The count is: 7
The count is: 8
End of program
```

Using else Statement with While Loop

Python supports to have an **else** statement associated with a loop statement.

- If the **else** statement is used with a **while** loop, the **else** statement is executed when the condition becomes false.

The following example illustrates the combination of an else statement with a while statement that prints a number as long as it is less than 5, otherwise else statement gets executed.

```
count = 0
while count < 5:
   print count, " is  less than 5"
   count = count + 1
else:
   print count, " is not less than 5"
```

When the above code is executed, it produces the following result −

0 is less than 5
1 is less than 5
2 is less than 5
3 is less than 5
4 is less than 5
5 is not less than 5

for loop

For loops are used for sequential traversal.
Syntax:
for var in sequence:
   statements(s)

Here, var is the variable that takes the value of the item inside the sequence on each iteration. Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation. It can be used to iterate over a range and iterators.

To loop through a set of code a specified number of times, we can use the range() function, The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example1:

```
for x in range(6):
  print(x)
```

Output:
0
1
2
3
4
5

Note that range (6) is not the values of 0 to 6, but the values 0 to 5.

The range () function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range (2, 6), which means values from 2 to 6 (but not including 6):

Example2:

```
n = 4
for i in range(0, n):
    print(i)
```

Output :
0
1
2
3

Example3:

```
for x in range(2, 6):
  print(x)
```

Output:
2
3
4
5

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, 3):

Example:

```
#Increment the sequence with 3 (default is 1):
for x in range(2, 30, 3):
  print(x, end= " ")
```

output:

2 5 8 11 14 17 20 23 26 29

Else in For Loop
The else keyword in a for loop specifies a block of code to be executed when the loop is finished:
Example: Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):
  print(x)
```

```
else:
  print("Finally finished!")
```

output:
0
1
2
3
4
5
Finally, finished

Loop Control Statements: Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

Continue Statement: It returns the control to the beginning of the loop.

Example:

```
for x in range(6):
  if x == 3: continue
  print(x)
```

output:
0
1
2
4
5

break statement: Break brings control out of the loop

Example1:

```
for x in range(6):
  if x == 3: break
  print(x)
```

output:
0
1
2

Example2:

```
for letter in 'GITAM UNIVERSITY':

        if letter == 'E' or letter == 'I':
         break
```

print ('Current Letter:', letter)

Output:
Current Letter: I

Example3:

```
for x in range(6):
    if x==3:break
    print(x)
else:
  print("Finally finished!")
```

Output:
0
1
2

Note: The else block will NOT be executed if the loop is stopped by a break statement.

Python pass Statement
In Python programming, the pass statement is a null statement which can be used as a placeholder for future code.
Suppose we have a loop or a function that is not implemented yet, but we want to implement it in the future. In such cases, we can use the pass statement.
The syntax of the pass statement is:
pass

Using pass With Conditional Statement

```
n = 10

# use pass inside if statement
if n > 10:
    pass

print('Hello')
```

Here, notice that we have used the pass statement inside the if statement .
However, nothing happens when the pass is executed. It results in no operation (NOP).